Hypothesis Testing with E-Values

Chapter 8: Handling Multiple E-Values

Based on Ramdas & Wang

Outline I

Why merge e-values at all? I

The setting. We have $K \geq 2$ e-variables $E_1, \ldots, E_K$, all valid for the *same* null hypothesis. Write $\mathcal{K} = \{1, \ldots, K\}$.

Where do multiple e-values come from?

- Several analysts (or several methods) each produce their own e-value for the same question.
- Data splitting or sequential updating produces one e-value per chunk of data (Chapters 5, 7).
- Multiple testing: one e-value per hypothesis (Chapter 9) – but this chapter is about combining e-values *for one and the same hypothesis*.

The recurring question of this chapter: given $E_1, \ldots, E_K$, how do we combine them into a single e-value $F(E_1, \ldots, E_K)$ that is itself valid – i.e. still has expectation at most 1 under the null? The answer depends critically on what we are willing to assume about the *dependence* between the E_k's.

Why merge e-values at all? II

Recall (from earlier chapters). A random variable $E \geq 0$ is an *e-variable* for a null hypothesis $\mathbb{P}$ if $\mathbb{E}^{\mathbb{P}}[E] \leq 1$ for every $\mathbb{P}$ in the null. Large values of E are evidence against the null; we reject when $E \geq 1/\alpha$, which controls the type-I error at level α by Markov's inequality.

A key selling point of e-values (versus p-values) is that they are *easy to combine* – often without any assumption on how they depend on each other. This chapter makes that promise precise: it tells you exactly which combination rules are valid, depending on what you know about the dependence between $E_1, \ldots, E_K$.

Notation you'll need

Merging function

A function $F : [0, \infty)^K \to [0, \infty)$ that takes K e-values as input and outputs a single combined value. We call F *valid* for a class of dependence structures if $F(E_1, \ldots, E_K)$ is guaranteed to be an e-variable whenever $E_1, \ldots, E_K$ are e-variables with that dependence structure.

Arbitrary dependence

We make *zero assumptions* about how $E_1, \ldots, E_K$ relate to one another. They could be independent, could be identical copies of each other, could be adversarially correlated by some process we don't understand – anything. A merging rule valid under arbitrary dependence must work no matter which of these is secretly true.

FWER: family-wise error rate

When testing K null hypotheses simultaneously and rejecting some subset, the FWER is $\mathbb{P}$(at least one rejected hypothesis is actually true). Controlling FWER at level α means this probability is $\leq \alpha$ no matter which hypotheses happen to be true.

Martingale merging function (preview)

8.1 Intuition: why averaging is the safe merge I

The puzzle. If $E_1, \ldots, E_K$ are each e-variables (each has mean ≤ 1 under the null), which combinations of them are still guaranteed to have mean ≤ 1 – with *no* assumption on their joint dependence?

Sum? No. $\mathbb{E}[E_1 + \cdots + E_K] = \mathbb{E}[E_1] + \cdots + \mathbb{E}[E_K] \leq K$, not ≤ 1. If all E_k happen to equal the same E, the sum is KE, which has mean up to K – badly invalid.
Max? No. Even two e-values with mean 1 each can be constructed so that $\max(E_1, E_2)$ has mean > 1 (concentrating mass on the event where at least one is large).
Product? Only under independence (Section 8.2) – under arbitrary dependence it can blow up just like the sum.
Average? Yes – always. By linearity of expectation alone (which needs *no* independence or dependence assumption whatsoever):

$$\mathbb{E}\left[\frac{E_1 + \cdots + E_K}{K}\right] = \frac{\mathbb{E}[E_1] + \cdots + \mathbb{E}[E_K]}{K} \leq \frac{1 + \cdots + 1}{K} = 1.$$

Linearity of expectation is the one identity in probability that *never cares about dependence*: $\mathbb{E}[X + Y] = \mathbb{E}[X] + \mathbb{E}[Y]$ whether X, Y are independent, identical, or adversarially entangled.

8.1 Intuition: why averaging is the safe merge II

Averaging exploits exactly this loophole – it is the unique "linear" merge that is immune to dependence, which is why it (and its weighted variants) turn out to be the *only* admissible choices in this fully-adversarial setting.

Definition 8.1: e-merging functions

E-merging function

An *e-merging function* (of K e-values) is an increasing Borel function

$$F : [0, \infty)^K \to [0, \infty)$$

such that, for *any* null hypothesis and *any* e-variables $E_1, \ldots, E_K$ for it (with arbitrary, unknown joint dependence), $F(E_1, \ldots, E_K)$ is again an e-variable.

- **Symmetric**: $F(\mathbf{e})$ is unchanged by permuting the coordinates of $\mathbf{e}$ (treats all K e-values interchangeably).
- **Dominates**: F dominates G if $F \geq G$ pointwise (strictly if also $F(\mathbf{e}) > G(\mathbf{e})$ somewhere). F is *admissible* if no e-merging function strictly dominates it – i.e. it can't be uniformly improved.
- **Essentially dominates**: F essentially dominates G if $G(\mathbf{e}) > 1 \implies F(\mathbf{e}) \geq G(\mathbf{e})$ – F is at least as good as G whenever G's output actually carries evidence (> 1).

The arithmetic mean and its weighted versions

Arithmetic mean $\mathbb{M}_K$

$$\mathbb{M}_K(e_1, \dots, e_K) = \frac{e_1 + \cdots + e_K}{K}, \qquad e_1, \dots, e_K \in [0, \infty).$$

Here $e_1, \dots, e_K$ are the realized e-values and K is how many there are. This is an e-merging function under arbitrary dependence (shown above).

Weighted arithmetic mean $\mathbb{M}_{\boldsymbol{\lambda}}$

For weights $\boldsymbol{\lambda} = (\lambda_1, \dots, \lambda_{K+1})$ in the simplex $\Delta_{K+1} = \{(x_1, \dots, x_{K+1}) \in [0,1]^{K+1} : \sum_k x_k = 1\}$,

$$\mathbb{M}_{\boldsymbol{\lambda}}(e_1, \dots, e_K) = \sum_{k=1}^{K} \lambda_k e_k + \lambda_{K+1}.$$

This mixes a weighted average of the e-values with a constant 1 (weight λ_{K+1} on "abstaining"); it is still an e-merging function under arbitrary dependence.

Geometric mean, harmonic mean, min: all valid too, but all are *dominated by* $\mathbb{M}_K$ (see below) – so there is never a reason to prefer them under arbitrary dependence.

Step-by-step: the average is an e-value, always I

Claim. If $E_1, \ldots, E_K$ are e-variables for a null $\mathbb{P}$ – with *any* joint dependence structure – then $\mathbb{M}_K(E_1, \ldots, E_K) = \frac{1}{K}\sum_k E_k$ is also an e-variable for $\mathbb{P}$.

Step 1 (what we're given). Each $E_k \geq 0$ and $\mathbb{E}^{\mathbb{P}}[E_k] \leq 1$. This is the *only* thing we know – we know nothing about $\mathrm{Cov}(E_i, E_j)$ or the joint law of $(E_1, \ldots, E_K)$.
Step 2 (nonnegativity of the average). Since each $E_k \geq 0$, the average $\frac{1}{K}\sum_k E_k \geq 0$ too. So it is a candidate e-variable (it's nonnegative, as required).
Step 3 (linearity of expectation – the crucial step). Expectation is linear *regardless of dependence*:

$$\mathbb{E}^{\mathbb{P}}\left[\frac{1}{K}\sum_{k=1}^{K} E_k\right] = \frac{1}{K}\sum_{k=1}^{K} \mathbb{E}^{\mathbb{P}}[E_k].$$

This identity holds whether the E_k are independent, identical, or adversarially dependent – linearity of expectation never needs an independence assumption.

Step-by-step: the average is an e-value, always II

Step 4 (plug in the bound). Using $\mathbb{E}^{\mathbb{P}}[E_k] \leq 1$ for each k,

$$\frac{1}{K}\sum_{k=1}^{K}\mathbb{E}^{\mathbb{P}}[E_k] \;\leq\; \frac{1}{K}\sum_{k=1}^{K}1 \;=\; \frac{K}{K} \;=\; 1.$$

Step 5 (conclude). Combining Steps 3–4, $\mathbb{E}^{\mathbb{P}}[\mathbb{M}_K(E_1,\ldots,E_K)] \leq 1$, and by Step 2 it is nonnegative. Hence $\mathbb{M}_K(E_1,\ldots,E_K)$ satisfies the definition of an e-variable for $\mathbb{P}$. Since $\mathbb{P}$ was an arbitrary null and $E_1,\ldots,E_K$ were arbitrary e-variables (arbitrary dependence included), $\mathbb{M}_K$ is a valid e-merging function. ■

Why this matters: no other step used any structural assumption about the joint distribution – the whole argument is "nonnegativity + linearity of expectation." That is exactly why averaging, uniquely among the natural combination rules, survives arbitrary dependence.

Proposition 8.3: the mean is (essentially) the best symmetric rule

Proposition 8.3

The arithmetic mean $\mathbb{M}_K$ essentially dominates any symmetric e-merging function: if a symmetric e-merging function G ever outputs a value > 1 (i.e. produces real evidence) at some input $\mathbf{e}$, then $\mathbb{M}_K(\mathbf{e}) \geq G(\mathbf{e})$.

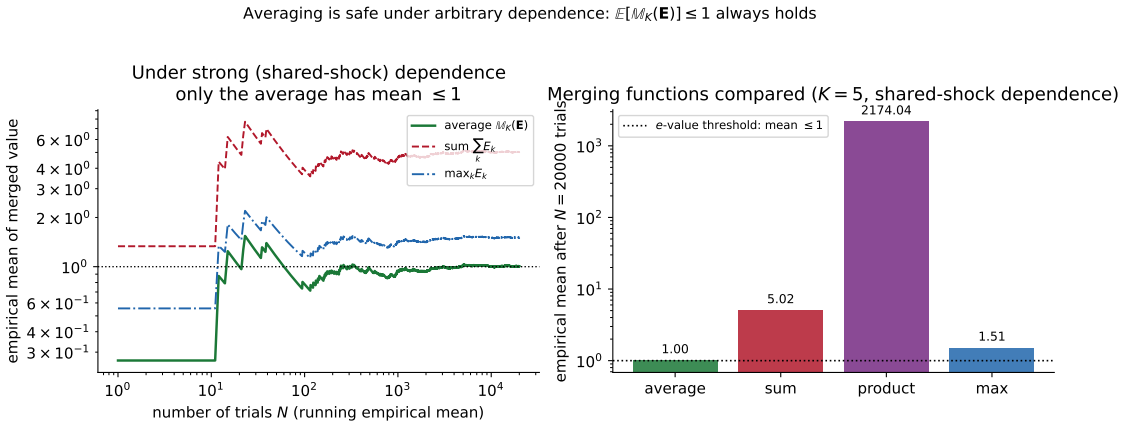
Proof sketch. Suppose some symmetric F beats $\mathbb{M}_K$ and also beats 1 at some point $\mathbf{e} = (e_1, \ldots, e_K)$: $b := F(\mathbf{e}) > \max(\mathbb{M}_K(\mathbf{e}), 1) =: a$. Let π be a uniformly random permutation of $\mathcal{K}$ and set $D_k = e_{\pi(k)}$ – a random reshuffling of the same numbers. Each D_k is itself an e-variable since $\mathbb{E}[D_k] = \mathbb{M}_K(\mathbf{e}) \leq a \leq 1$ after a suitable independent thinning by an event A with $\mathbb{P}(A) = 1/a$. By symmetry of F, $\mathbb{E}[F(D'_1, \ldots, D'_K)] \geq b/a > 1$ – contradicting that F is a valid e-merging function. Hence no such F exists.

Theorem 8.4 (complete characterization). F is an *admissible* e-merging function if and only if $F = \mathbb{M}_{\boldsymbol{\lambda}}$ for some $\boldsymbol{\lambda} \in \Delta_{K+1}$. In words: *every* sensible way to merge arbitrarily dependent e-values is (dominated by) a weighted average with a constant.

Python: averaging survives adversarial dependence

```python
import numpy as np

rng = np.random.default_rng(0)
K, p, N = 5, 0.1, 20000

# Build K e-values that are STRONGLY DEPENDENT: they all move together
# via a shared latent shock W (fires with probability p).
a = np.array([8.0, 6.0, 10.0, 5.0, 9.0])      # values when shock hits
b = (1 - p * a) / (1 - p)                      # baseline (chosen so E[e_k]=1)

W = rng.binomial(1, p, size=N)
E = np.where(W[:, None] == 1, a, b)            # shape (N, K), all rows dependent

avg  = E.mean(axis=1)      # average across the K e-values, per trial
summ = E.sum(axis=1)
prod = E.prod(axis=1)

print("empirical mean of average:", avg.mean())   # ~1.00  (valid e-value!)
print("empirical mean of sum:    ", summ.mean())  # ~K     (invalid)
print("empirical mean of product:", prod.mean())  # >>1    (invalid, dependent)
```

Figure: averaging is safe, sum/product are not



Under a shared-shock dependence structure ($K = 5$, each E_k individually has mean exactly 1), the empirical mean of the *average* converges to ≤ 1 (in fact $= 1$), while the sum and product drift far above 1 – they are not valid e-merging functions here.

8.2 Intuition: independence buys you the product

Averaging was safe because it only used linearity of expectation. If we are additionally willing to assume $E_1, \ldots, E_K$ are *independent*, we unlock a stronger tool: the expectation of a *product* of independent random variables factors:

$$\mathbb{E}[E_1 \cdots E_K] = \mathbb{E}[E_1] \cdots \mathbb{E}[E_K] \leq 1 \cdots 1 = 1.$$

This is the e-value analogue of Fisher's method for combining independent p-values. Under independence, the product is not just valid – it is, in a precise sense, the "biggest" valid merge (Proposition 8.7 below). But bigger mean comes at a cost, which is exactly the subject of Section 8.4.

Definition 8.5: ie-merging functions

Ie-merging function

An *ie-merging function* is a Borel function $F : [0,\infty)^K \to [0,\infty)$ such that $F(E_1,\dots,E_K)$ is an e-variable whenever $E_1,\dots,E_K$ are *independent* e-variables (for any null). Unlike e-merging functions, F need not be increasing in every coordinate.

Product Π_K

$$\Pi_K(e_1,\dots,e_K) = \prod_{k=1}^{K} e_k, \qquad e_1,\dots,e_K \in [0,\infty).$$

Proposition 8.6 (a simple admissibility check)

If F is increasing and Borel and $\mathbb{E}^{\mathbb{P}}[F(\mathbf{E})] = 1$ for *every* vector of independent *exact* e-variables $\mathbf{E}$ (exact meaning $\mathbb{E}[E_k] = 1$, not just ≤ 1), then F is an admissible ie-merging function.

Since $\mathbb{E}[E_1 \cdots E_K] = \prod_k \mathbb{E}[E_k] = 1$ for independent exact e-variables, Π_K satisfies this and is admissible.

U-statistics: interpolating between mean and product

U-statistic merging functions

For $n \in \{0, 1, \ldots, K\}$,

$$U_n(e_1, \ldots, e_K) = \frac{1}{\binom{K}{n}} \sum_{A \subseteq \mathcal{K} : |A| = n} \left(\prod_{k \in A} e_k \right).$$

Here U_n averages the size-n subset-products over all $\binom{K}{n}$ subsets A of size n.

- $n = K$: only one subset ($A = \mathcal{K}$), so $U_K = \Pi_K$, the full product.
- $n = 1$: averages single e-values, so $U_1 = \mathbb{M}_K$, the arithmetic mean.
- $n = 0$: the empty product is 1, so $U_0 \equiv 1$ (the trivial, always-valid e-value).
- Convex mixtures $\sum_n c_n U_n$ ($c_n \geq 0$, $\sum_n c_n = 1$) are also admissible ie-merging functions (by Prop. 8.6).

Not a complete class: Example 8.8 in the book exhibits an admissible ie-merging function, $f(e_1, e_2) = \frac{1}{2}\left(\frac{e_1}{1+e_1} + \frac{e_2}{1+e_2}\right)(1 + e_1 e_2)$, that is *not* any convex mixture of the U_n's – so, unlike the arbitrary-dependence case, there is no simple complete characterization here.

Proposition 8.7: the product is (weakly) optimal

Proposition 8.7

The product Π_K weakly dominates any ie-merging function F: for all $(e_1, \dots, e_K) \in [1, \infty)^K$ (i.e. whenever *every* input already carries some evidence, $e_k \geq 1$),

$$F(e_1, \dots, e_K) \leq \Pi_K(e_1, \dots, e_K) = e_1 \cdots e_K.$$

Why only on $[1, \infty)^K$? With a large K, it is rare that *all* inputs exceed 1 simultaneously, so this is a genuinely weak requirement – much weaker than full domination on all of $[0, \infty)^K$.

Beyond independence: negative dependence also works. If $E_1, \dots, E_K$ are *negative lower orthant dependent* (NLOD), meaning

$$\mathbb{P}\Big(\bigcap_{k \in \mathcal{K}} \{E_k \leq x_k\} \Big) \leq \prod_{k \in \mathcal{K}} \mathbb{P}(E_k \leq x_k) \quad \text{for all } (x_1, \dots, x_K),$$

then Π_K (and the U_n's) remain valid e-merging functions even though $E_1, \dots, E_K$ are not independent – NLOD is a weaker, one-directional relaxation of independence (equality in (8.5) recovers exact independence).

Python: 8.2 – product needs independence

```python
import numpy as np
rng = np.random.default_rng(1)

K, N = 5, 20000

# INDEPENDENT e-values: each E_k = 2*Bernoulli(1/2), mean = 1 exactly.
E_indep = 2 * rng.binomial(1, 0.5, size=(N, K))
prod_indep = E_indep.prod(axis=1)
print("independent product, empirical mean:", prod_indep.mean())   # ~1.0 (valid)

# DEPENDENT e-values built as identical COPIES: E_k = E for all k.
E_shared = 2 * rng.binomial(1, 0.5, size=(N, 1))
E_copies = np.repeat(E_shared, K, axis=1)
prod_copies = E_copies.prod(axis=1)
print("copied (dependent) product, empirical mean:", prod_copies.mean())
# ~ 2^{K-1}, badly invalid -- independence was doing all the work above
```

8.3 Intuition: e-values that arrive one at a time

Independence is a strong, static assumption about K e-values considered jointly. A more flexible and dynamic notion: e-values that arrive *sequentially*, where each new one is only required to be "fair" (mean ≤ 1) *given everything seen so far* – exactly like a fair bet on the next round of a game, regardless of how earlier rounds went. This generalizes independence (independent e-values are automatically sequential) and connects directly to the testing-by-betting framework of Chapter 7.

Definition 8.9: sequential e-values, se-merging functions

Sequential e-values

$E_1, \ldots, E_K$ are *sequential* for a class of nulls $\mathcal{P}$ if, for every $\mathbb{P} \in \mathcal{P}$ and every $k \in \mathcal{K}$,

$$\mathbb{E}^{\mathbb{P}}[E_k \mid E_1, \ldots, E_{k-1}] \leq 1.$$

That is: conditional on everything observed before round k, the next e-value E_k is still (conditionally) a valid e-variable. This is strictly more general than independence.

Se-merging function

A Borel $F : [0, \infty)^K \to [0, \infty)$ such that $F(E_1, \ldots, E_K)$ is an e-variable for *any* sequential e-variables $E_1, \ldots, E_K$. *Exact* if $F(\mathbf{E})$ is exact (mean $= 1$) whenever the inputs are exact and sequential.

All convex mixtures of the U-statistics U_n (including $\mathbb{M}_K$ and Π_K) are se-merging functions, since independent e-values are automatically sequential.

Definition 8.10: martingale merging functions

Martingale merging function

F is a *martingale merging function* if

$$F(e_1, \ldots, e_K) = \prod_{k=1}^{K} (1 - \lambda_k + \lambda_k e_k),$$

where $\lambda_1 \in [0, 1]$ is a fixed constant and, for $k \geq 2$, $\lambda_k = \lambda_k(e_1, \ldots, e_{k-1}) \in [0, 1]$ is allowed to depend on the e-values seen *before* round k.

Betting interpretation. Think of capital that starts at 1. At round k, you bet a *fraction* λ_k of your current capital on the outcome E_k: if the bet were fully committed you'd multiply by e_k, but betting only a fraction λ_k of capital gives the factor $(1 - \lambda_k) + \lambda_k e_k$ (keep $1 - \lambda_k$ untouched, multiply λ_k by e_k). Multiplying these factors across rounds tracks total capital.

- $\lambda_k \equiv 1/K$ for all k recovers (a variant connected to) the arithmetic mean – betting a *fixed amount*, not a fixed proportion, each round.
- $\lambda_k \equiv 1$ for all k recovers the product Π_K – "all-in" every round.

Proposition 8.11 and Theorem 8.12

Proposition 8.11

Any martingale merging function is an exact se-merging function. Moreover, a convex combination of martingale merging functions is again a martingale merging function.

Proof idea (backward induction). Write $L_k = \lambda_k(E_1, \dots, E_{k-1})$. Conditioning on the past up to round $K-1$:

$$\mathbb{E}^{\mathbb{P}}[F(E_1, \dots, E_K) \mid \mathcal{F}_{K-1}] = F(E_1, \dots, E_{K-1}, 1)\big(1 - L_K + L_K\, \mathbb{E}^{\mathbb{P}}[E_K \mid \mathcal{F}_{K-1}]\big) \le F(E_1, \dots, E_{K-1}, 1),$$

using $\mathbb{E}^{\mathbb{P}}[E_K \mid \mathcal{F}_{K-1}] \le 1$ and $L_K \in [0,1]$. Peel off rounds $K, K-1, \dots, 1$ this way to get $\mathbb{E}^{\mathbb{P}}[F(E_1, \dots, E_K)] \le F(1, \dots, 1) = 1$.

Theorem 8.12 (completeness)

Every se-merging function is dominated by *some* martingale merging function. So, just as $\mathbb{M}_{\boldsymbol{\lambda}}$ completely characterizes admissible merges under arbitrary dependence, martingale merging functions completely characterize (up to domination) valid merges of sequential e-values.

Examples: hit-and-stop, and the empirically adaptive process

Example 8.13: hit-and-stop

Fix a level $\alpha \in (0,1)$. As soon as the running combined value $F(e_1, \dots, e_k, 1, \dots, 1)$ first reaches $1/\alpha$ (i.e. we could already reject), set all future $\lambda_j = 0$ – stop betting. This makes F *non-monotonic* in general (an early win can be "locked in" rather than risked later), which is perfectly fine: martingale merging functions need not be increasing, only *sequentially* monotone (increasing in e_k holding future e-values at their neutral value 1).

Example 8.14: empirically adaptive e-process

Choose λ_k at each round by optimizing the log-growth of capital *so far*:

$$\lambda_k(e_1, \dots, e_{k-1}) = \operatorname*{arg\,max}_{\lambda \in [0,\gamma]} \frac{1}{k-1} \sum_{s=1}^{k-1} \log\big((1-\lambda) + \lambda e_s\big), \qquad \gamma \in (0,1].$$

A data-driven, default choice with no prior knowledge of the alternative required – and it is asymptotically log-optimal (Theorem 7.22).

Python: 8.3 – a martingale merging function in action

```python
import numpy as np

def martingale_merge(e, lam):
    """F(e_1,...,e_K) = prod_k (1 - lam_k + lam_k * e_k)."""
    e = np.asarray(e, dtype=float)
    lam = np.asarray(lam, dtype=float)
    return np.prod(1 - lam + lam * e)

e = [0.8, 1.3, 2.1, 0.5, 1.8]            # our running example (see next slide)

# Fixed-fraction betting, lambda_k = 1/K for all k (mean-like betting):
lam_fixed = [1/5]*5
print("fixed 1/K betting:", martingale_merge(e, lam_fixed))

# All-in every round (lambda_k = 1), i.e. this equals the raw product:
lam_allin = [1.0]*5
print("all-in (= product):", martingale_merge(e, lam_allin), "vs prod:", np.prod(e))
```

Running example: 5 correlated tests of the SAME coin

Setup. We suspect one coin is unfair, and we run $K = 5$ different tests of the very same coin (e.g. 5 different statisticians, or 5 different test statistics, applied to the same sequence of flips). Because they all use the *same underlying coin and data*, the resulting e-values are genuinely, unavoidably **dependent** – there is no reason to assume independence here.

$$e_1 = 0.8, \quad e_2 = 1.3, \quad e_3 = 2.1, \quad e_4 = 0.5, \quad e_5 = 1.8.$$

Average (by hand):

$$\mathbb{M}_5(\mathbf{e}) = \frac{0.8 + 1.3 + 2.1 + 0.5 + 1.8}{5} = \frac{6.5}{5} = 1.3.$$

Product (by hand):

$$\Pi_5(\mathbf{e}) = 0.8 \times 1.3 \times 2.1 \times 0.5 \times 1.8 = 1.9656.$$

Numerically the product (1.9656) is larger than the average (1.3) – more "evidence," at first glance.

Running example: which merge is actually valid here?

- **The average, 1.3, is a valid e-value regardless of the dependence between the 5 tests.** By Section 8.1's argument (linearity of expectation only), $\mathbb{M}_5(\mathbf{E})$ has mean ≤ 1 under the null no matter how the 5 tests on the same coin are correlated. We may safely report 1.3 as combined evidence and, e.g., reject at level α if $1.3 \geq 1/\alpha$ (here, $\alpha \leq 1/1.3 \approx 0.77$ – weak evidence, appropriately modest given the redundancy across 5 dependent tests of one coin).
- **The product, 1.9656, is not justified here.** Section 8.2's guarantee for the product requires independent (or at least NLOD) e-values. Since all 5 tests examine the very same coin and data, treating them as independent would double-count the same evidence 5 times over – the product could then wildly overstate the evidence (recall Section 8.1's dependence simulation, where an analogous product's empirical mean blew up past 2000).

Takeaway

Numerically, product $>$ average here ($1.9656 > 1.3$), but only the average carries a valid statistical guarantee under the (realistic) dependence of this example. Reporting the product would be an invalid, overconfident use of e-value theory.

Python: the running example, verified

```python
import numpy as np

e = np.array([0.8, 1.3, 2.1, 0.5, 1.8])
K = len(e)

avg  = e.mean()          # valid regardless of dependence (Section 8.1)
prod = e.prod()          # valid ONLY if the 5 tests are independent (Section 8.2)

print(f"average = {avg}")    # 1.3
print(f"product = {prod}")   # 1.9656

alpha_avg = 1/avg
print(f"average rejects for alpha >= {alpha_avg:.3f}")    # alpha >= 0.769

# The product looks stronger, but on the SAME coin's data the 5 tests are
# dependent, so we have NO guarantee that E[prod] <= 1 -- only the
# average carries a distribution-free guarantee here.
```

8.4 Intuition: bigger mean is not free

Proposition 8.7 says the product Π_K weakly dominates every ie-merging function – so isn't the product simply "the best" choice under independence? **No.** Under the alternative hypothesis $\mathbb{Q}$ (where we *want* large e-values), the product does have the largest possible expectation among se-merging functions applied to "powered" e-values ($\mathbb{E}^{\mathbb{Q}}[E_k] \geq 1$) – but a large mean can be driven almost entirely by a vanishingly rare, enormous outcome, while the *typical* realization is disappointing (even 0). This is the classic mean-variance trade-off: the product maximizes the mean but also (weakly) maximizes the variance among exact se-merging functions.

Proposition 8.16: moments are maximized by the product

Lemma 8.15

If F is an ie-merging function and $a \geq 1$, then $G(e_1, \ldots, e_K) = F(ae_1, e_2, \ldots, e_K)/a$ is also an ie-merging function. (Rescaling one coordinate by $a \geq 1$ and dividing the output by a preserves validity.)

Proposition 8.16

Let F be an se-merging function. For independent, nonnegative $E_1, \ldots, E_K$ under an alternative $\mathbb{Q}$ with $\mathbb{E}^{\mathbb{Q}}[E_k] \geq 1$ for each k, and for every moment order $m \in [1, \infty)$:

$$\mathbb{E}^{\mathbb{Q}}\big[F(E_1, \ldots, E_K)^m\big] \;\leq\; \prod_{k=1}^{K} \mathbb{E}^{\mathbb{Q}}[E_k^m] \;=\; \mathbb{E}^{\mathbb{Q}}\big[\Pi_K(E_1, \ldots, E_K)^m\big].$$

In particular (taking $m = 2$, exact e-variables), $\mathrm{var}^{\mathbb{P}}(F(\mathbf{E})) \leq \mathrm{var}^{\mathbb{P}}(\Pi_K(\mathbf{E}))$: **the product has the largest variance among all exact se-merging functions.**

So: the product maximizes *every* moment (mean, variance, everything) among se-merging functions applied to independent, powered e-values – but a large variance is generally undesirable, since it means the outcome is unreliable.

Example 8.17: all-in versus log-optimal

Suppose (under $\mathbb{Q}$) each E_k, $k = 1, 2, \ldots$, independently takes value 4 w.p. $1/2$ and 0 w.p. $1/2$ (so $\mathbb{E}^{\mathbb{Q}}[E_k] = 2$, a genuinely powered e-value).

- **All-in strategy** ($\lambda_k \equiv 1$, i.e. the raw product): $M_k^{\text{all-in}} = \Pi_k(E_1, \ldots, E_k) = 4^k$ w.p. 2^{-k}, and $= 0$ w.p. $1 - 2^{-k}$. Its mean $\mathbb{E}^{\mathbb{Q}}[M_k^{\text{all-in}}] = 2^k$ is the *largest possible* – but $M_k^{\text{all-in}} \to 0$ almost surely as $k \to \infty$: the strategy is a near-certain wipeout that happens to have an astronomically lucky tail.
- **Log-optimal strategy** ($\lambda_k \equiv 1/3$, the growth-rate maximizer): M_k^* grows like $(4/3)^k$ almost surely (law of large numbers) – slower in expectation, but reliable: it doesn't collapse to 0.

Moral

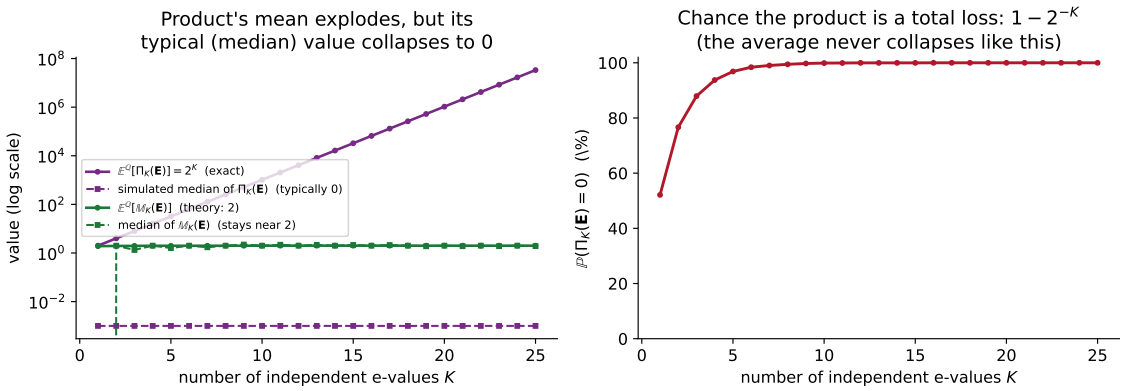
E-power (Chapter 7) is measured by the expected *logarithm* under $\mathbb{Q}$, not by any raw moment. The all-in/product strategy wins on every moment yet loses on the metric that actually matters for a useful, repeatable procedure.

Python: 8.4 – product vs. average, mean-variance trade-off

```python
import numpy as np
rng = np.random.default_rng(2)

N, Kmax = 4000, 25
draws = rng.binomial(1, 0.5, size=(N, Kmax))      # 1 = "high" outcome (e_k = 4)
e_vals = np.where(draws == 1, 4.0, 0.0)           # each e_k has mean 2

for K in [1, 5, 10, 25]:
    e_K = e_vals[:, :K]
    prod_K, avg_K = e_K.prod(axis=1), e_K.mean(axis=1)
    theory_mean_prod = 2.0**K                          # exact, not simulated
    print(f"K={K:2d}  E[prod] (exact)={theory_mean_prod:.3g}  "
          f"median(prod)={np.median(prod_K):.3g}  "
          f"mean(avg)={avg_K.mean():.3g}  P(prod=0)={np.mean(prod_K==0)*100:.1f}%")
# The product's mean explodes exponentially, yet its typical (median)
# value is 0 with probability -> 1: the average is the "reliable" merge.
```

Figure: the mean-variance trade-off

Mean-variance trade-off (Section 8.4): product = high mean, high risk; average = modest mean, reliable



Left: the product's exact mean 2^K (theory) explodes exponentially, while its simulated *median* collapses to (essentially) 0; the average's mean and median both stay near 2. Right: the probability that the product is a total loss ($= 0$) tends to 1 as K grows – a risk the average never carries.

8.5 Intuition: from merging to multiple testing

Now suppose $E_1, \dots, E_K$ are e-values for K *different* null hypotheses $\mathcal{P}_1, \dots, \mathcal{P}_K$ (rather than K e-values for one hypothesis). We want a testing procedure that outputs a set of rejected hypotheses $\mathcal{D} \subseteq \mathcal{K}$ while controlling the **family-wise error rate (FWER)**: the chance of even a single false rejection, however many hypotheses are actually true.

$$\mathbb{P}(\mathcal{D} \cap \mathcal{N} \neq \varnothing) \leq \alpha \quad \text{for every possible } \mathbb{P},$$

where $\mathcal{N} = \{k \in \mathcal{K} : \mathbb{P} \in \mathcal{P}_k\}$ is the (unknown) set of hypotheses that happen to be true. The merging tools from 8.1–8.4 turn out to give a simple, general recipe for this.

The mean e-collection and its FWER guarantee

For any subset $A \subseteq \mathcal{K}$, merge the e-values of hypotheses in A into a single e-value for the *intersection null* $\bigcap_{k \in A} \mathcal{P}_k$. Using the arithmetic mean (valid under arbitrary dependence – we don't know how $E_1, \ldots, E_K$ relate across hypotheses):

Mean e-collection (8.11)

$$E_A = \frac{1}{|A|} \sum_{k \in A} E_k, \qquad A \subseteq \mathcal{K}.$$

E_A is a valid e-variable for $\bigcap_{k \in A} \mathcal{P}_k$ for every subset A simultaneously (by the Section 8.1 argument, applied separately to each A).

The e-testing procedure

Define $E_k^* = \min_{A \subseteq \mathcal{K}: k \in A} E_A$ (the smallest merged value over all subsets containing k), and reject k whenever $E_k^* \geq 1/\alpha$.

Why this controls FWER: a two-line argument

Claim: the rule "reject k iff $E_k^* \geq 1/\alpha$" controls FWER at level α.

Step 1. For any true null index $k \in \mathcal{N}$ and *any* $A \ni k$ with A possibly restricted to $\mathcal{N}$: since $A \subseteq \mathcal{N}$ is available as a candidate (in particular $A = \mathcal{N}$), $E_k^* = \min_{A \ni k} E_A \leq E_{\mathcal{N}}$. Taking the max over $k \in \mathcal{N}$:

$$\max_{k \in \mathcal{N}} E_k^* \;\leq\; E_{\mathcal{N}}.$$

Step 2 (Markov's inequality). $E_{\mathcal{N}}$ is a genuine e-variable for $\bigcap_{k \in \mathcal{N}} \mathcal{P}_k$ (the truth), so $\mathbb{P}(E_{\mathcal{N}} \geq 1/\alpha) \leq \alpha \cdot \mathbb{E}[E_{\mathcal{N}}] \leq \alpha$.

Conclusion.

$$\mathbb{P}(\mathcal{D} \cap \mathcal{N} \neq \varnothing) = \mathbb{P}\Big(\max_{k \in \mathcal{N}} E_k^* \geq 1/\alpha \Big) \leq \mathbb{P}(E_{\mathcal{N}} \geq 1/\alpha) \leq \alpha.$$

This works for *any* e-merging function used to build E_A (we only ever need E_A to be *some* valid e-variable for $\bigcap_{k \in A} \mathcal{P}_k$ – it need not even be computed from $E_1, \ldots, E_K$ directly; such a general family $(E_A)_{A \subseteq \mathcal{K}}$ is called an *e-collection*).

Algorithm 8.18: computing E_k^* in $O(K^2)$

Naively, computing $E_k^* = \min_{A \ni k} E_A$ requires checking 2^{K-1} subsets A per index k. For the mean e-collection there is a fast shortcut using order statistics.

Key fact

Sort $e_1, \dots, e_K$ ascendingly as $e_{(1)} \le \cdots \le e_{(K)}$ via a permutation π (so $e_{(j)} = e_{\pi(j)}$), and let $S_i = e_{(1)} + \cdots + e_{(i)}$. Then

$$E^*_{\pi(k)} = \min_{i \in [k-1]} \frac{e_{\pi(k)} + S_i}{i+1},$$

i.e. the best subset containing index $\pi(k)$ is always formed by adding the smallest available e-values to it – so we only need to scan prefixes of the sorted list, not all 2^{K-1} subsets.

Complexity. Sorting costs $O(K \log K)$; for each of the K indices we scan up to K prefixes, giving $O(K^2)$ overall (Algorithm 8.18). If instead we use the product ie-merging function (and assume independence across hypotheses), an $O(K)$ algorithm is available.

Python: 8.5 – FWER control via the mean e-collection

```python
import numpy as np

def e_star_mean_collection(e):
    """O(K^2) computation of E_k^* for the mean e-collection (Algorithm 8.18)."""
    e = np.asarray(e, dtype=float)
    K = len(e)
    order = np.argsort(e)                    # ascending order statistics
    e_sorted = e[order]
    S = np.concatenate([[0.0], np.cumsum(e_sorted)])   # S[i] = sum of i smallest

    e_star = np.empty(K)
    for k in range(K):
        idx = order[k]
        best = e[idx]                        # subset A = {idx} alone, i=0
        for i in range(1, k + 1):            # add the i smallest OTHER values
            candidate = (e[idx] + S[i]) / (i + 1)
            best = min(best, candidate)
        e_star[idx] = best
    return e_star

e = np.array([0.3, 1.2, 6.0, 0.7, 9.5])    # 5 e-values, 5 different hypotheses
alpha = 0.1
e_star = e_star_mean_collection(e)
rejected = np.where(e_star >= 1/alpha)[0]
print("E*_k:", np.round(e_star, 3))
print("rejected hypotheses (index):", rejected)    # FWER <= alpha, guaranteed
```

Chapter 8 summary

- **8.1 Arbitrary dependence:** only (weighted) arithmetic means are admissible e-merging functions. The proof is pure linearity of expectation – no independence needed. Sum, max, product can all fail badly.
- **8.2 Independence:** the product Π_K is a valid, weakly-optimal ie-merging function; more generally so are U-statistic mixtures, and even negatively dependent (NLOD) e-values allow the product.
- **8.3 Sequential e-values:** martingale merging functions $\prod_k (1 - \lambda_k + \lambda_k e_k)$, with a betting interpretation, completely characterize valid merges (up to domination) when e-values arrive one at a time and each is conditionally fair given the past.
- **8.4 Mean-variance trade-off:** the product maximizes every moment (mean, variance, ...) among se-merging functions, but this large variance means it is fragile – it can collapse to 0 with high probability even while its mean is enormous. E-power (expected log) is the metric that actually matters, and it favors less aggressive merges.
- **8.5 FWER control:** merge e-values per subset $A \subseteq \mathcal{K}$ (e.g. via the mean) to get E_A; reject k when $E_k^* = \min_{A \ni k} E_A \geq 1/\alpha$. A two-line Markov argument shows this controls FWER at level α, for any valid e-merging rule.

References

Based on Chapter 8 of A. Ramdas and R. Wang, *Hypothesis Testing with E-values*, arXiv:2410.23614. The chapter's results are mainly drawn from Vovk and Wang [2021, 2024a] (e-merging functions, Theorem 8.12), Block et al. [1982] (negative lower orthant dependence) and Chi et al. [2024] (dependence results for merging), and Vovk and Wang [2021] / Hartog and Lei [2025] (FWER-controlling e-value procedures), with connections to closed testing in Xu et al. [2025] and Goeman et al. [2025].